The other option is to store the data on the server. It requires more work to set up and is rarely necessary. First, Rails has to keep track of sessions. It does this by creating (by default) a 32-hex character key (which means there are 16^{32} possible combinations). This key is called the *session ID*, and it's effectively random. Rails arranges to store this session ID as a cookie (with the key `_session_id`) on the user's browser. Because subsequent requests come into the application from this browser, Rails can recover the session ID.

Second, Rails keeps a persistent store of session data on the server, indexed by the session ID. When a request comes in, Rails looks up the data store using the session ID. The data that it finds there is a serialized Ruby object. It deserializes this and stores the result in the controller's `session` attribute, where the data is available to our application code. The application can add to and modify this data to its heart's content. When it finishes processing each request, Rails writes the session data back into the data store. There it sits until the next request from this browser comes along.

What should you store in a session? You can store anything you want, subject to a few restrictions and caveats:

- Some restrictions apply on what kinds of object you can store in a session. The details depend on the storage mechanism you choose (which we'll look at shortly). In the general case, objects in a session must be serializable (using Ruby's `Marshal` functions). This means, for example, that you can't store an I/O object in a session.

- If you store any Rails model objects in a session, you'll have to add `model` declarations for them. This causes Rails to preload the model class so that its definition is available when Ruby comes to deserialize it from the session store. If the use of the session is restricted to just one controller, this declaration can go at the top of that controller.

```
class BlogController < ApplicationController
```